3. Generics in Java — Quick Notes

Generics

Purpose

- Write code once
- Use for any non-primitive datatype

Primitive vs Non-Primitive

Primitive	Non-Primitive
int, char, float	Classes/Objects
No inheritance	Inherit from Object

Wrapper Classes

Primitive	Wrapper
int	Integer
char	Character

Why Generics?

Avoid Writing Same Code Repeatedly

Examples:

- add(Integer, Integer)
- add(Double, Double)

Instead: Write one generic function

Generics in Collections

Collections are generic: ArrayList, LinkedList, Stack, Vector, Set, Map

Example: ArrayList<Integer>

Same class works for:

- Integer
- Character
- Student
- Any object type

Generics Can Be

Type	Example
Generic Class	ArrayList<T>
Generic Interface	List<T>
Generic Method	Collections.sort()

Generic Methods

Collections methods are generic: **sort(), binarySearch(), max(), min()**
Single implementation works for all datatypes.

Problem Without Generics

Using Object: Object x;

Problem

- Not type-safe
- Runtime exceptions possible

Example:

- ClassCastException

Advantage of Generics

Type Safety

Errors caught at: Compile time ✅
Instead of: Runtime ❌

Generic Class Syntax

class Pair<T, S>

- T → datatype of x
- S → datatype of y

Example: Pair<Integer, String>

Type Inference (Java 1.7+)

Instead of: new Pair<Integer, String>()
Can write: new Pair<>()

Compiler infers types automatically.

Generic Function Syntax: <T> int count(T arr[], T x)

Important: Datatype parameters written: Before return type

Multiple Generic Types: <T, S>

Supports multiple datatype parameters.

Important Point

For generic methods:

- Compiler infers datatype automatically
- No need to specify explicitly

Internal Working Insight

```
class MyGen<T> {
   static int count = 0;

   MyGen() {
      count++;
   }
}
MyGen<Integer> m1 = new MyGen<>();
MyGen<String> m2 = new MyGen<>();

System.out.println(MyGen.count);
```

Output: 2

Important

- Only ONE class created internally
- Different from C++ templates

Static variable shared.